BOOTCAMP IA – P8 MADRID 2026

TDD en Producción:
La Red de Seguridad

De escribir código con miedo a diseñar soluciones con confianza.

El Cambio de Mentalidad

Enfoque Tradicional

1. Escribir código.
2. Ejecutar la app.
3. Buscar errores manualmente.
4. Rezar en el despliegue.

Mentalidad TDD

1. Diseñar la expectativa (Test).
2. Escribir código mínimo.
3. Validar automático.
4. Limpiar con seguridad.

El Latido del Desarrollo

Tres pasos iterativos. Segundos, no minutos.



Escribe un Test

Define qué quieres lograr. Falla por diseño.

Haz que Pase

Escribe el código más simple posible.

Refactor

Mejora el diseño. Elimina duplicidad.

Ejemplo de trabajo con TDD

Principales ventajas aportadas.

1

Escribir el Test (Red)

En esta fase se escribe una prueba para validar una funcionalidad específica, que al inicio siempre fallará. Esto asegura que el código implementado se alinea con los requerimientos.

Ejemplo práctico: Si estás desarrollando una calculadora, podrías escribir un test para verificar que suma(2,3) devuelve 5. Este test inicialmente fallará porque aún no existe la función suma

Fase 1:
Escribir el test (**Red**)
def test_suma():
assert suma(2, 3) == 5

2

Implementar la funcionalidad mínima (Green)

El siguiente paso consiste en escribir el código mínimo para superar la prueba.

En el **ejemplo**, la implementación de suma (a,b) que devuelva a+b será suficiente.

Fase 2:
Implementar la funcionalidad mínima (**Green**)
def suma(a, b):
return a + b

3

Refactorizar (Refactor)

Tras superar la prueba, se procede a optimizar el código. Este paso permite mejorar la eficiencia y legibilidad sin alterar la funcionalidad.

Fase 1:
Refactorizar (**Refactor**)
def suma(a, b):
Posibilidad de agregar validación de entradas en el futuro
return a + b

Herramienta de Diseño

Diseño por Contrato

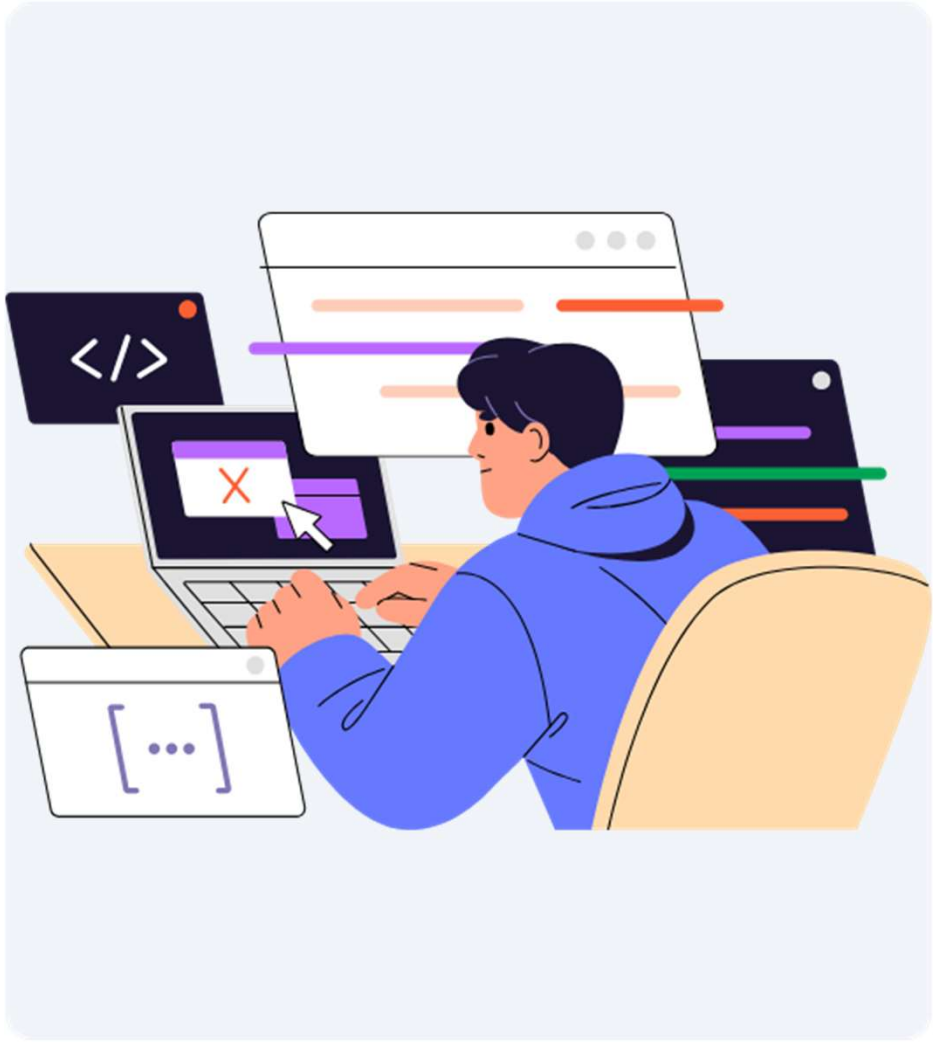
Decides cómo se usará tu función antes de crearla.

Desacoplamiento Natural

Código difícil de testear = Código mal diseñado.

Foco Absoluto

Resuelves un solo problema a la vez.



Beneficios clave del Test Driven Development

Principales ventajas aportadas.

1

Mejor calidad de código

El TDD obliga a los desarrolladores a pensar en casos de uso antes de escribir código, garantizando que el diseño sea modular, claro y enfocado a las necesidades reales

2

Facilita el mantenimiento

Una base sólida de pruebas reduce la probabilidad de errores al realizar cambios en el código. Esto facilita la escalabilidad y mantenimiento del software.

3

Mejora la colaboración en equipo

Gracias a las pruebas claras y bien definidas, los equipos de desarrollo, testers y stakeholders pueden trabajar con una mejor comunicación y alineación de objetivos.

Costes y beneficios del TDD

Principales aspectos

Aspecto	Coste Inicial	Beneficio a largo plazo
Tiempo de desarrollo	Mas lento al escribir pruebas antes del código	Menor tiempo en depuración y corrección de errores
Curva de aprendizaje	Formación del equipo en metodologías y herramientas	Mayor eficiencia en el diseño y la implementación del código
Calidad del código	Inversión inicial para escribir pruebas detalladas	Código modular, fácil de escalar y mantener

El Impacto Real en Producción

Por qué los equipos maduros no despliegan sin pruebas.

Documentación Viva

Tests legibles que nunca se desactualizan.

Refactor Seguro

Modifica arquitectura base sin miedo a romper.

Menos Debugging

Errores capturados en local, no en servidores.

BEST PRACTICES

Los Principios F.I.R.S.T.

Fast

Ejecución en milisegundos. Feedback inmediato.

Independent

Cero dependencias. Orden de ejecución irrelevante.

Repeatable

Mismos resultados en cualquier entorno o servidor.

Self-Validating

Pasa o falla. Sin intervención ni revisión manual.

Timely

Escritos justo antes del código de producción.

TDD en el Sprint Ágil

Conectando requerimientos de negocio con código funcional.

1

Historia de Usuario

Definición del valor para el negocio.

2

Criterios de Aceptación

Traducción técnica. Base de los tests.

3

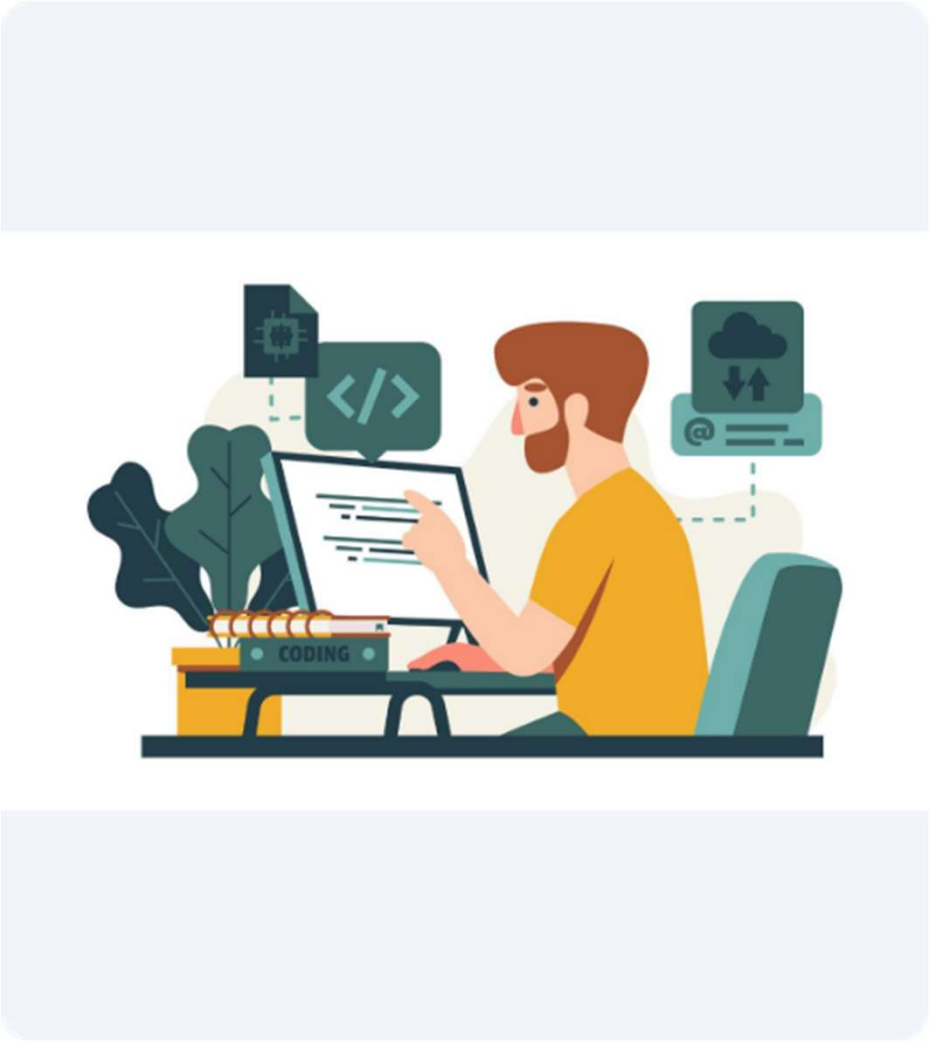
Ciclo TDD

Red-Green-Refactor continuo.

4

Done

Funcionalidad lista y protegida.



ANTI-PATRONES

Trampas al Empezar

Testear la Implementación

Valida el resultado final, no las variables internas.

Pasos Demasiado Grandes

Escribir 50 líneas antes de ejecutar el test.

Olvidar el Refactor

Dejar el primer código verde que funcionó.

El código profesional no se prueba al final.

Empieza pequeño. Escribe el test primero. Construye con confianza.

Para aprender más:

https://www.ibm.com/es-es/think/topics/test-driven-development

Source: ibm.com

---

https://es.wikipedia.org/wiki/Desarrollo_guiado_por_pruebas

Source: Wikipedia.org